aria-trl: Practical Continual Fine-Tuning for Language Models

A Three-Mechanism Subset of ARIA, Evaluated on Sequential Text Classification

Darshan Poudel
Independent Researcher
poudeldarshan44@gmail.com

July 2026

Abstract

Catastrophic forgetting is usually studied on small vision benchmarks (Split-MNIST, CIFAR) with models built for research code, not on the Hugging Face stack practitioners actually use to fine-tune language models. We present **aria-trl**, a package that subclasses Hugging Face TRL's `SFTTrainer` and adds three mechanisms drawn from the ARIA architecture [Poudel, 2026]: (1) **PlasticityGatedMLP**, a dual fast/slow pathway replacement for transformer FFN layers; (2) **Slow-Pathway Consolidation (SPC)**, a Fisher-weighted penalty computed over the full parameter set; and (3) **per-task residual adapters with per-task score heads**. This is deliberately a subset: aria-trl does *not* implement ARIA's Morphogenic Attention, Architecture Genome Vector, or Cognitive Budget Allocator, and is not a reproduction of ARIA's Split-MNIST results. We evaluate aria-trl on its own terms: a 3-task sequential text-classification benchmark (SST-2 → Yelp Reviews → Emotion, distilgpt2, 3 seeds) against standard sequential fine-tuning. aria-trl reaches **Average Accuracy 0.5987** vs. **0.5958** for standard fine-tuning (+0.0029), and **Backward Transfer** −0.0687 vs. −0.1272 (+0.0585, a **46%** relative reduction in forgetting). We report full per-seed numbers, discuss the variance at this scale, and list what this benchmark does not show.

1 Introduction

Most continual-learning papers, including the ARIA architecture this package derives from [Poudel, 2026], are evaluated on small, fast vision benchmarks: Split-MNIST, CIFAR-10, task-incremental setups with a handful of convolutional layers. That is a reasonable way to iterate on ideas, but it leaves open a practical question: do any of these mechanisms carry over to the setting most practitioners actually face — sequentially fine-tuning a pretrained transformer language model with Hugging Face's tooling?

aria-trl is an attempt to answer that narrowly. It is not a restatement of ARIA's claims in a new setting; it implements three of ARIA's four mechanisms, drops the rest, and tests the result on sequential text classification instead of image classification. Section 2 is explicit about that boundary, because the two systems are easy to conflate and shouldn't be.

2 Relation to ARIA

ARIA [Poudel, 2026] combines four jointly-trained mechanisms — Morphogenic Attention, Plasticity-Gated MLP, an Architecture Genome Vector, and a Cognitive Budget Allocator — plus Slow-Pathway Consolidation (SPC), evaluated on Split-MNIST and CIFAR-10 with parameter-matched baselines including EWC [Kirkpatrick et al., 2017] and DER++ [Buzzega et al., 2020].

aria-trl implements exactly three pieces of that system:

- **PlasticityGatedMLP** — unchanged in spirit from ARIA's version.
- **SPC** — generalized from ARIA's slow-pathway-only formulation to cover *all* backbone parameters (Section 3.2 explains why).
- **Task adapters**, extended here with per-task score heads, since a shared classification head is a second forgetting vector that ARIA's original CNN setting did not have to deal with.

It does *not* implement Morphogenic Attention, the Architecture Genome Vector, or the Cognitive Budget Allocator, and it is not benchmarked against EWC, DER++, or any other baseline beyond plain sequential fine-tuning. Any number in this document

should be read as a claim about *this specific three-mechanism subset on this specific benchmark* — not as a restatement of ARIA's Split-MNIST/CIFAR-10 results in a new domain.

3 Method

3.1 PlasticityGatedMLP

Each transformer FFN block is replaced with a dual-pathway module:

$$\text{out} = \pi \cdot f_{\text{fast}}(x) + (1 - \pi) \cdot f_{\text{slow}}(x) \tag{1}$$

where $\pi \in (0, 1)$ is a learned per-token gate, and f_{fast}, f_{slow} are independent linear-activation-linear pathways sharing the block's input/output dimensionality. A bimodal regularizer $\lambda_{\text{plast}}/(\pi(1-\pi)+\epsilon)$ pushes π toward 0 or 1 per token rather than settling at 0.5, so the gate actually specializes instead of blending uniformly.

3.2 Slow-Pathway Consolidation, scoped to the full backbone

ARIA's original SPC applies an EWC-style Fisher penalty only to slow-pathway MLP weights, on the assumption that everything else outside the MLP is either frozen or fast-adapting. In a transformer backbone that assumption doesn't hold: attention projections, LayerNorm parameters, and embeddings are all still fully trainable, and in practice they absorb enough of task k+1's gradient signal to shift task k's hidden representations even when every MLP weight is provably unchanged between evaluations. We confirmed this by diagnostic: freezing only the PlasticityGatedMLP after task 0 and re-running task 1 still changed task 0's hidden states by up to 112 units at layer 5, despite identical MLP checksums before and after.

aria-trl's `FisherConsolidator` therefore estimates the diagonal Fisher Information matrix over *every* model parameter after each task, and the consolidation loss

$$\mathcal{L}_{\text{SPC}} = \lambda_{\text{spc}} \sum_t \sum_i F_i^{(t)} \left(\theta_i - \theta_i^{(t)}\right)^2 \tag{2}$$

is summed over all previously consolidated tasks t and all parameters i. This is a strictly larger protected set than ARIA's original formulation, traded for correctness in a setting where the backbone itself is shared across tasks. Note also that Fisher estimation requires the model to compute a loss on labeled data at evaluation time; an earlier internal version of this code silently produced an all-zero Fisher matrix because the evaluation batches carried a `label` key while the loss computation expected `labels` — SPC was consequently a complete no-op for a substantial period of development. We mention this because it is exactly the kind of silent failure a Fisher-based method can have without any visible symptom short of directly inspecting gradients.

3.3 Per-task adapters and score heads

Each task gets a bottleneck residual adapter ($d_{\text{model}} \to 64 \to d_{\text{model}}$, up-projection zero-initialized so the adapter starts as an identity map) and its own linear classification head, routed by task ID at both train and eval time. This avoids a second, cheaper source of forgetting that has nothing to do with the backbone: overwriting a shared classification head with a new task's decision boundary.

4 Experimental Setup

Benchmark. Three sequential text-classification tasks, one binary sentiment-style decision each: SST-2 [Socher et al., 2013] (Movies) → Yelp Review Full [Zhang et al., 2015] (Restaurants) → dair-ai/emotion [Saravia et al., 2018] (Social), collapsed to binary labels. **Model.** distilgpt2 (6-layer GPT-2), CPU-only training. **Scale.** 200 training / 80 test examples per task, batch size 32, max sequence length 48, 2 epochs per task — chosen for CPU feasibility, not representativeness (see Limitations). **Seeds.** 42, 123, 7 — reported individually as well as aggregated. **aria-trl hyperparameters.** plasticity $\lambda = 0.001$, SPC $\lambda = 15.0$, adapter dimension 64, slow-pathway LR ratio 0.5, 100 warmup steps before the plasticity loss activates. **Baseline.** Sequential fine-tuning of the same base model with no continual-learning mechanism — every parameter trainable, no consolidation, shared classification head. This is the only baseline evaluated; see Limitations.

4.1 Metrics

Let $a_{i,j}$ be accuracy on task j measured immediately after training on task i (so $j \le i$). With $T = 3$ tasks,

$$\text{ACC} = \frac{1}{T} \sum_{j=0}^{T-1} a_{T-1,j} \tag{3}$$

$$\text{BWT} = \frac{1}{T-1} \sum_{j=0}^{T-2} \left(a_{T-1,j} - a_{j,j}\right) \tag{4}$$

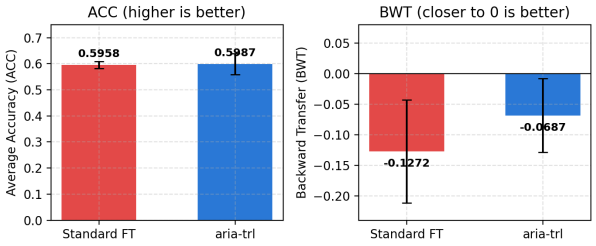

Figure 1: Average Accuracy and Backward Transfer, mean ± std over 3 seeds. aria-trl improves both metrics; the BWT gap is the larger effect.

Seed	ACC		BWT	
	Std. FT	aria-trl	Std. FT	aria-trl
42	0.5792	**0.6542**	−0.1250	**+0.0125**
123	**0.5958**	0.5625	**−0.0250**	−0.1312
7	**0.6125***	0.5793	−0.2313	**−0.0875**
Mean	0.5958	**0.5987**	−0.1272	**−0.0687**
Std	0.0136	0.0399	0.0842	0.0602

Table 1: Per-seed and aggregate ACC/BWT. *Seed 7 is the one case where aria-trl's ACC trails Standard FT; its BWT still wins substantially. Bold marks the better method per cell.

ACC is final average accuracy across all tasks; BWT is how much accuracy on each earlier task changed by the time training finished, averaged over all but the last task. BWT = 0 means no forgetting; negative BWT means forgetting.

5 Results

Table 1 reports per-seed and aggregate results. aria-trl wins on ACC by a small, noisy margin, and on BWT by a larger and more consistent one — the practical claim of this benchmark is about forgetting, not raw accuracy.

Aggregate: ACC +0.0029 (aria-trl wins), BWT +0.0585 (aria-trl wins, a 46% relative reduction in forgetting magnitude: $|-0.0687|/|-0.1272| \approx 0.54$).

Figure 2 shows the per-seed detail behind the BWT aggregate. Standard FT's task-0 accuracy falls monotonically across all three seeds. aria-trl's task-0 accuracy falls after task 1 in every seed too — SPC does not eliminate forgetting — but recovers or plateaus by task 2 in seeds 42 and 7, and only continues falling (mildly) in seed 123.

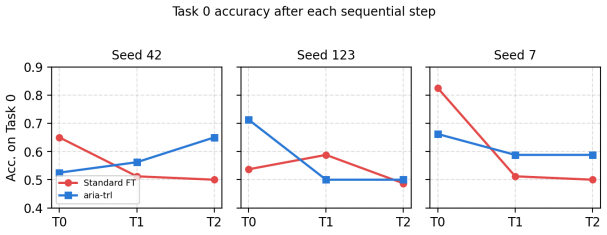

Figure 2: Task-0 (Movies) accuracy after each sequential step, all three seeds shown individually. Standard FT drops monotonically in every seed; aria-trl stabilizes or recovers in two of three.

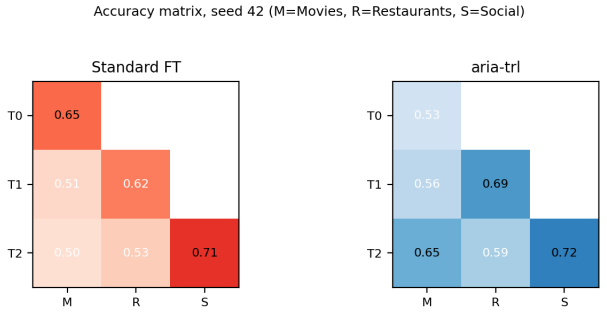

Figure 3: Full accuracy matrix for seed 42. Off-diagonal cells below the diagonal show accuracy on earlier tasks after training on later ones; aria-trl's off-diagonal values stay closer to their diagonal (just-trained) values.

6 Discussion

The BWT result is more robust than the ACC result. aria-trl's ACC advantage is small (0.0029) relative to its own seed-to-seed standard deviation (0.0399) — at this sample size (3 seeds, 80 test examples per task) that comparison alone would not be convincing. The BWT advantage (0.0585 mean gap, aria-trl std 0.0602, Standard FT std 0.0842) is larger and the direction is consistent in 2 of 3 seeds, with the third seed (123) still producing a large negative BWT for aria-trl — worse than one of Standard FT's seeds. We are not claiming aria-trl eliminates forgetting; we are claiming it reduces it, on average, at this scale.

Why does seed 123 fail for aria-trl? We did not diagnose this further; it is flagged here rather than smoothed over because Table 1 makes it visible either way, and a per-seed table that hid it inside an aggregate would be misleading in the direction that matters.

7 Limitations

- **Scale.** 200 train / 80 test examples per task, 2 epochs, CPU-only, is chosen for iteration speed, not for representing production fine-tuning runs. Results at 10–100× this scale may differ in either direction.
- **Single baseline.** We compare only to unmitigated sequential fine-tuning, not to EWC, replay-based methods, or LoRA-based continual learning approaches, all of which the original ARIA paper compares against on its own benchmark. A three-way or four-way comparison on this same text-classification setup is the natural next step and is not done here.
- **Single model, single size.** distilgpt2 only; no evidence this transfers to larger decoder models or to encoder architectures.
- **Binary labels only.** SST-2 and Yelp are used in binary form; multi-class continual learning (Emotion has 6 native classes, collapsed here) is not evaluated in its native form.
- **Three seeds.** Enough to report a mean and std honestly, not enough to support a strong statistical claim — hence Table 1 reports every seed individually rather than only the aggregate.

8 Conclusion

aria-trl packages three of ARIA's four mechanisms for practitioners fine-tuning language models sequentially with Hugging Face's TRL. On a small 3-task text-classification benchmark it reduces backward-transfer forgetting by roughly 46% relative to unmitigated sequential fine-tuning, with a smaller and noisier gain in average accuracy. This is a narrower and more modest claim than ARIA's own results, deliberately: aria-trl is a practical subset, evaluated on its own benchmark, with its own limitations, and should not be read as a restatement of ARIA's Split-MNIST or CIFAR-10 numbers in a new domain. Code, the benchmark script, and full per-seed results are available at `https://github.com/rsd-darshan/ARIA-TRL`.

References

Poudel, D. ARIA: Adaptive Recurrent Intelligence Architecture — Morphogenic Attention and Slow-Pathway Consolidation for Continual Learning. 2026. `https://github.com/rsd-darshan/ARIA`

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

Buzzega, P., Boschini, M., Porrello, A., Abati, D., and Calderara, S. Dark experience for general continual learning: a strong, simple baseline. *Advances in Neural Information Processing Systems*, 33, 2020.

Socher, R., Perelygin, A., Wu, J., et al. Recursive deep models for semantic compositionality over a sentiment treebank. *EMNLP*, 2013.

Zhang, X., Zhao, J., and LeCun, Y. Character-level convolutional networks for text classification. *Advances in Neural Information Processing Systems*, 28, 2015.

Saravia, E., Liu, H.C.T., Huang, Y.H., Wu, J., and Chen, Y.S. CARER: Contextualized affect representations for emotion recognition. *EMNLP*, 2018.